A3-PS015-TC317

ExplainableHomeandLast-Mile Connectivity Diagnosis

TEAM : NetDetect
CH. Sarita - saritachagantipati@aceec.ac.in
CH. Sujit - sujitchakka10@gmail.com
A. Pavan Sai - pavansaiaitha@gmail.com

AIORI Hackathon 3
Ace Engineering College - ECE
October 5, 2026

Problem Description

Technical Challenges

1. Internet speed alone does not explain every connectivity problem.
2. High latency and packet loss can degrade real-time applications even when throughput appears adequate.
3. DNS failures can prevent access to websites by name even when IP connectivity remains available.
4. Congestion may increase latency and jitter when the network is under load.
5. Users often lack clear technical evidence to share with network support teams.

Project Objective

- To develop a guided desktop diagnostic prototype that measures network behavior, identifies likely connectivity issues, explains the evidence behind each finding, and generates an escalation report when the cause cannot be isolated.

Background

Internet connectivity depends on several components, including the user's device, local network gateway, DNS resolver, access network, and remote destination.
Different network metrics describe different aspects of connectivity:

- Latency: The round-trip time for a test packet or request, typically measured in milliseconds.
- Packet loss: The percentage of transmitted probes for which no response is received within the defined timeout.
- DNS resolution delay: The time required to resolve a domain name into an IP address.
- Throughput: The rate at which data is successfully transferred, usually expressed in bits per second.
- Jitter or delay variation: Variation in packet delay, which can affect real-time applications.
- Loaded latency: Latency measured while the connection is carrying additional traffic.

Literature & RFC Review

1. RFC 2330 — Framework for IP Performance Metrics :

RFC 2330 provides a framework for defining and collecting IP network performance metrics. It discusses measurement methodology, samples, statistics, and the importance of clearly defined metrics.

Application to this project:

- Define what each network test measures.
- Record measurement conditions and results consistently.
- Avoid making unsupported claims from isolated measurements.
- Implement DNS resolution checks.
- Record successful resolutions and failures.
- Measure DNS lookup duration and handle exceptions.
- Implement DNS resolution checks.
- Record successful resolutions and failures.

Proposed Solution

Overall Solution

Theapplication will guide users through a series of network tests, collect measurable evidence, apply transparent diagnostic rules, and generate a report explaining the likely cause of a connectivity problem.

Main Components

1. Desktop Diagnostic Interface
2. Network Measurement Engine
3. Explainable Decision Engine
4. Controlled Fault-Injection
5. Evidence-Based Reporting

Expected Outcome

A working prototype that demonstrates guided diagnostics, transparent decision rules, controlled fault testing, and evidence-based escalation reporting.

Optimization Proposed

Area	Conventional approach	Proposed approach
Performance assessment	Mainly relies on speed-test results	Combines throughput, latency, loss, DNS, and reachability
DNS troubleshooting	May be overlooked	Explicit DNS resolution checks
Congestion analysis	May rely on subjective observations	Compares idle and loaded measurements
Diagnosis explanation	May provide only a generic error	Shows evidence and the rule behind each finding
Testing reliability	Often depends on uncontrolled conditions	Uses repeatable, controlled fault scenarios

Solution Architecture & Design

Solution architecture: probes, rules, and evidence

Desktop guidance → controlled experiments → measurements → diagnosis or escalation.

Desktop agent

Guides the user through the test sequence and presents the explanation.

Controlled fault-injection lab

Introduces known gateway, DNS, packet-loss, and congestion conditions.

Measurement probes

Collect reachability, DNS response, latency, packet loss, and throughput.

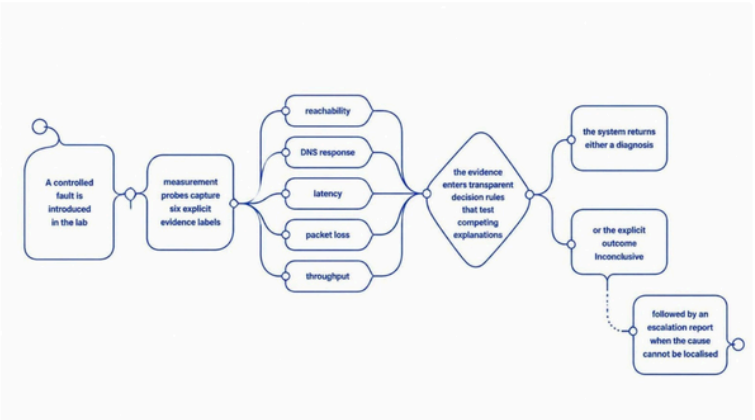
Explanation and reporting engine

Applies transparent rules; produces a diagnosis or escalation report.

Data Flow Diagram

Data flow: from fault injection to escalation

Every conclusion remains traceable to the measurements that support it.



Deployment Timeline

- Sprint 1: Set upPython, GitHub, and basic network tests.
- Sprint 2: Develop diagnostic rules and error handling.
- Sprint 3: Test the four controlled network faults.
- Sprint 4: Integrate the desktop UI, reports, and final demo.

Development Methodology

- Build the project in small, incremental stages.
- Use modular Python code.
- Diagnose problems using measurable evidence.
- Validate all four faults in a controlled lab.
- Test results and handle inconclusive cases.
- Use GitHub for collaboration and version control.

Team Responsibilities

- Member 1: Network tests and fault simulation.
- Member 2: Desktop UI, diagnosis, and reports.
- Both: Integration, testing, and demonstration.

Bibliography

[1] RFC 2330: Framework for IP Performance Metrics Measurement framework and shared terminology

[2] PS-015: Explainable Home and Last-Mile Connectivity Diagnosis Project brief, pilot scope,fault scenarios, and delivery requirements

Thank You for Your Attention!